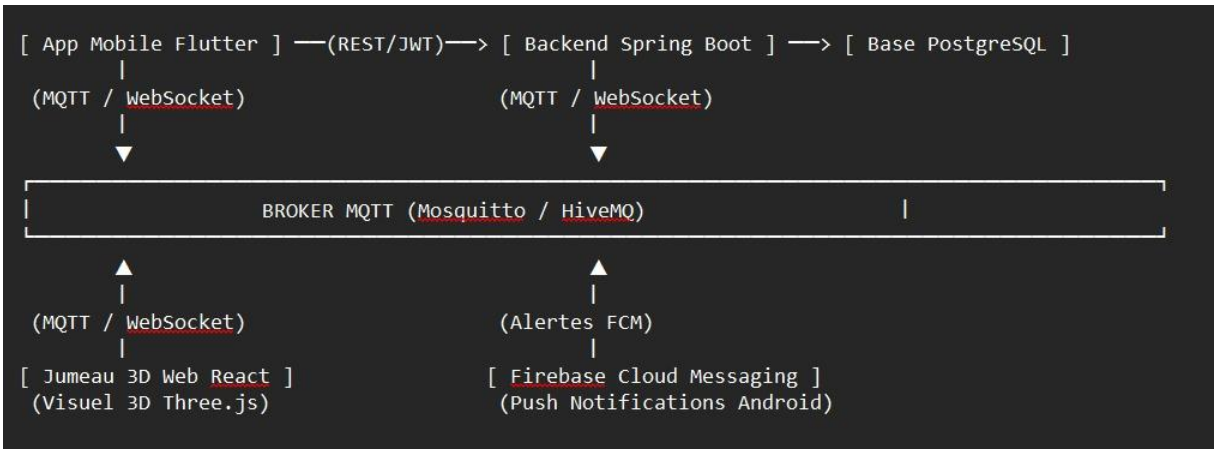
Document architectural

1. Vue d'ensemble de l'architecture globale du projet

Le projet repose sur une architecture moderne en couches, séparant clairement la partie mobile, la communication IoT et le serveur backend :



2. Structure globale du projet Flutter (*Features-First*)

Côté mobile, l'application est découpée par fonctionnalités métier dans lib/features/, avec un dossier core/ pour tout ce qui est partagé.

```
lib/
├── core/                          # Éléments partagés globaux
│   ├── constants/                 # Constantes (URLs API, broker MQTT)
│   ├── local_storage/             # Gestion de Hive (token JWT, cache local)
│   ├── theme/                     # Thème de l'application
│   └── widgets/                   # Widgets globaux (bannières d'alerte, boutons communs)
│
├── features/                      # Modules métiers de l'application
│   ├── auth/                      # Module d'authentification (Écran 4)
│   ├── dashboard/                 # Module Accueil / Contrôle des appareils (Écran 1)
│   ├── surveillance/              # Module Sécurité, capteurs & alertes (Écran 2)
│   └── automations/               # Module Scénarios & Routines (Écran 3)
│
├── app.dart                       # Configuration globale et routes
└── main.dart                      # Point d'entrée et initialisation (Hive, etc.)
```

3. Zoom détaillé : L'intérieur d'un dossier de fonctionnalité important

Feature Auth :

```
features/
└── auth/
    ├── data/
    │   ├── datasources/
    │   │   └── auth_api_client.dart       # Appels d'authentification vers Spring Boot (login/signup)
    │   ├── models/
    │   │   └── user_model.dart            # Modèle JSON (from/to JSON pour l'utilisateur et le token)
    │   └── repositories_impl/
    │       └── auth_repository_impl.dart  # Implémentation concrète de l'authentification
    │
    ├── domain/
    │   ├── entities/
    │   │   └── user_entity.dart           # Objet utilisateur pur (id, email, role)
    │   ├── repositories/
    │   │   └── auth_repository.dart       # Contrat abstrait (login(), register(), logout())
    │   └── usecases/
    │       ├── login_usecase.dart         # Logique métier pour connecter l'utilisateur
    │       └── get_profile_usecase.dart   # Récupérer les infos du profil
    │
    └── presentation/
        ├── providers/
        │   └── auth_provider.dart         # Gestion d'état de la session (connecté / déconnecté)
        ├── screens/
        │   ├── login_screen.dart          # Écran de connexion (Écran 4)
        │   └── profile_screen.dart        # Écran de gestion du profil
        └── widgets/
            └── auth_text_field.dart       # Champs de texte stylisés pour les formulaires
```

Features Dashboard :

```
features/
└── dashboard/
    │
    ├── data/                       # 1. COUCHE DONNÉES (Connexion externe & technique)
    │   ├── datasources/            # Appels bruts (ex: mqtt_service.dart, api_client.dart)
    │   ├── models/                 # Modèles de données JSON avec fromJson/toJson
    │   │   └── device_model.dart   # Ex: Représentation de l'appareil reçu de l'API
    │   └── repositories_impl/      # Implémentation concrète des contrats du domaine
    │       └── device_repository_impl.dart
    │
    ├── domain/                     # 2. COUCHE MÉTIER / CŒUR DE L'APPLI (Pur Dart, zéro dépendance UI/Réseau)
    │   ├── entities/               # Objets métiers purs
    │   │   └── device_entity.dart  # Ex: Propriétés de l'appareil (id, nom, état allumé/éteint)
    │   ├── repositories/           # Contrats / Interfaces abstraites
    │   │   └── device_repository.dart # Ex: getDevices(), toggleDevice()
    │   └── usecases/               # Actions métiers unitaires déclenchées par l'utilisateur
    │       └── toggle_device_usecase.dart # Ex: Logique métier pour basculer l'état d'un appareil
    │
    └── presentation/               # 3. COUCHE INTERFACE (UI, Écrans & Gestion d'état)
        ├── providers/              # Gestion d'état (Provider / Riverpod)
        │   └── dashboard_provider.dart # Ex: Met à jour l'écran quand un appareil change d'état
        ├── screens/                # Écrans principaux
        │   └── dashboard_screen.dart # Ex: La vue globale du tableau de bord
        └── widgets/                # Widgets visuels spécifiques à cette feature
            └── device_card.dart    # Ex: La carte graphique d'une ampoule ou d'un volet
```

Feature Surveillance :

```
features/
└── surveillance/
    ├── data/
    |   ├── datasources/
    |   |   └── surveillance_remote_datasource.dart # Récupération des logs d'incidents depuis l'API
    |   ├── models/
    |   |   └── alert_model.dart           # Modèle de données d'une alerte (fumée, intrusion)
    |   └── repositories_impl/
    |       └── surveillance_repository_impl.dart
    |
    ├── domain/
    |   ├── entities/
    |   |   └── alert_entity.dart          # Entité pure d'une alerte (niveau de gravité, date, capteur)
    |   ├── repositories/
    |   |   └── surveillance_repository.dart
    |   └── usecases/
    |       └── get_alert_logs_usecase.dart # Récupérer l'historique des alertes de Neon DB
    |
    └── presentation/
        ├── providers/
        |   └── surveillance_provider.dart # Gère les alertes en temps réel reçues par MQTT/Push
        ├── screens/
        |   └── surveillance_screen.dart   # Écran principal de sécurité (Écran 2)
        └── widgets/
            └── alert_tile.dart            # Ligne visuelle d'une alerte (ex: "Détecteur de fumée - Cuisine")
```

Feature Automations :

```
features/
└── automations/
    ├── data/
    |   ├── datasources/
    |   |   └── automation_api_client.dart # Enregistrement des scénarios auprès du backend
    |   ├── models/
    |   |   └── scenario_model.model.dart  # Modèle JSON d'une règle (Si / Alors)
    |   └── repositories_impl/
    |       └── automation_repository_impl.dart
    |
    ├── domain/
    |   ├── entities/
    |   |   └── scenario_entity.dart       # Entité pure d'un scénario (titre, conditions, statut actif/inactif)
    |   ├── repositories/
    |   |   └── automation_repository.dart
    |   └── usecases/
    |       ├── toggle_scenario_usecase.dart # Activer ou désactiver une routine
    |       └── create_scenario_usecase.dart # Créer une nouvelle règle personnalisée
    |
    └── presentation/
        ├── providers/
        |   └── automations_provider.dart  # État de la liste des scénarios
        ├── screens/
        |   └── automations_screen.dart    # Écran de gestion des routines (Écran 3)
        └── widgets/
            └── scenario_card.dart         # Carte interactive pour activer/désactiver un scénario
```

Pourquoi cette organisation est puissante :

- **Isolation :** Si vous changez de Framework de state management ou de bibliothèque réseau, le cœur métier (domain) et le visuel (presentation) ne sont pas cassés.
- **Clarté :** Tout ce qui concerne le contrôle des équipements de la maison se trouve regroupé au même endroit, ce qui évite de s'éparpiller dans tout le projet.

4. Structure globale du Backend Spring Boot (*Layered Architecture*)

Côté serveur, le backend est structuré de manière classique et robuste pour dialoguer avec la base de données Neon DB et le broker MQTT :

```
src/main/java/com/safehome/
|
├── SafeHomeApplication.java        # Point d'entrée Spring Boot
|
├── config/                         # Configurations (Sécurité JWT, MQTT, WebSocket, HikariCP)
||
├── controller/                     # Couche Web (Expose les API REST pour Flutter)
|   ├── AuthController.java         # Connexion & Inscription
|   ├── DeviceController.java       # Gestion des états des équipements
|   └── ScenarioController.java     # Gestion des règles et scénarios
|
├── service/                        # Couche Métier (Logique applicative et IoT)
|   ├── DeviceService.java          # Traitement des commandes
|   ├── MqttService.java            # Gestion des publications/abonnements MQTT
|   └── NotificationService.java    # Envoi des alertes via Firebase Cloud Messaging (FCM)
|
├── repository/                     # Couche d'accès aux données (Spring Data JPA)
|   ├── UserRepository.java
|   ├── DeviceRepository.java
|   └── LogRepository.java          # Historique des capteurs et alertes
|
├── model/                          # Entités JPA (Tables PostgreSQL / Neon DB)
|   ├── User.java
|   ├── Device.java
|   └── EventLog.java
|
└── dto/                            # Objets de transfert de données (Requêtes / Réponses API)
    ├── request/
    └── response/
```

5. Résumé des flux de communication

1. **Flutter →\rightarrow→ Spring Boot (REST) :** Pour l'authentification (JWT) et la récupération initiale des configurations.
2. **Flutter ↔\leftrightarrow↔ MQTT ↔\leftrightarrow↔ React 3D :** Pour le temps réel. Quand l'utilisateur clique sur une lumière dans l'app mobile, un message MQTT (home/salon/light/set) est envoyé au broker. Le jumeau 3D (React) écoute ce même canal et anime la maison virtuelle instantanément.
3. **Spring Boot →\rightarrow→ Firebase →\rightarrow→ Flutter :** Lorsqu'un capteur critique se déclenche (fumée dans la cuisine ou fuite d'eau), le backend analyse l'incident, enregistre le log en base de données (Neon DB) et pousse une notification d'urgence sur le smartphone de l'utilisateur.